DOCUMENTACAO TECNICA

Deck of Cards API para um app Blackjack em Flutter

Funcionalidades, endpoints, significado dos dados e estruturas Dart para integracao.

Projeto	Aplicativo Blackjack
API	Deck of Cards API
Base URL	https://deckofcardsapi.com
Plataforma cliente	Flutter / Dart
Versao do documento	1.0 - 21/09/2026

Escopo: documentacao do contrato HTTP publicado em deckofcardsapi.com e recomendacoes de absorcao dos dados no aplicativo. A API fornece baralho, cartas e pilhas; as regras de Blackjack permanecem no app.

1 Visao geral

A Deck of Cards API e um servico HTTP que simula baralhos de cartas. Ela cria baralhos ordenados ou embaralhados, compra cartas, reorganiza o restante e administra pilhas nomeadas. Para Blackjack, a integracao recomendada cria um baralho com seis decks, compra cartas para jogador e dealer e usa pilhas para representar as maos.

LIMITE DE ESCOPO	A API nao calcula pontuacao, nao identifica Blackjack, nao decide vencedor, nao controla apostas e nao aplica regras como hit, stand, double ou split. Essas responsabilidades pertencem ao dominio do aplicativo Flutter.

Conteudo do documento

2 Contrato HTTP e convencoes
3 Catalogo de endpoints
4 Criacao, compra e reembaralhamento
5 Operacoes com pilhas
6 Dicionario de dados
7 Codigos de cartas e regras de interpretacao
8 Estruturas Dart para o Flutter
9 Cliente HTTP e tratamento de erros
10 Fluxo sugerido para Blackjack
11 Testes, cuidados e referencias

Resumo das funcionalidades

Grupo	O que a API permite	Uso no Blackjack
Baralho	Criar um ou varios decks, ordenados ou embaralhados; criar subconjunto de cartas; incluir curingas.	Iniciar o shoe e reiniciar uma partida.
Compra	Retirar uma ou mais cartas do topo; criar e comprar de um deck novo na mesma chamada.	Distribuir cartas ao jogador e dealer.
Reembaralhamento	Embaralhar todas as cartas ou somente as que ainda estao no monte principal.	Renovar o shoe sem perder cartas que estao em maos/pilhas.
Pilhas	Criar, adicionar, listar, embaralhar e retirar cartas do topo, fundo ou aleatoriamente.	Representar maos, descarte ou areas do jogo.
Retorno	Devolver cartas compradas ou cartas de uma pilha ao monte principal.	Encerrar rodada e reaproveitar cartas quando a regra do app permitir.
Imagens	URLs PNG/SVG da frente e PNG do verso.	Renderizar as cartas no tabuleiro.

2 Contrato HTTP e convencoes

Item	Definicao
Base URL	https://deckofcardsapi.com
Formato	JSON para operacoes de baralho/pilha; imagem para recursos em /static/img/.
Metodo	Os exemplos oficiais usam GET. A documentacao informa que certos parametros de criacao tambem podem ser enviados por POST; para Flutter, GET e suficiente e mais direto.
Autenticacao	Nao informada/exigida na documentacao oficial.
Identificador	deck_id identifica o baralho no servidor. Um baralho sem atividade por duas semanas e descartado pelo servico.
Estado remoto	Cada chamada altera ou consulta o estado associado ao deck_id; ele deve ser persistido durante a partida.
Codificacao	Parametros ficam na query string. Nomes de pilhas devem ser tratados como segmentos de URL e codificados com Uri.

Parametros recorrentes

Parametro	Tipo	Obrigatorio	Significado e validacao
deck_id	String (path)	Sim, exceto /new/	ID devolvido na criacao. O valor especial new cria um baralho embaralhado no endpoint de compra.
deck_count	int	Nao	Quantidade de decks combinados. Padrao: 1. Para Blackjack, a documentacao sugere 6.
count	int	Nao/depende	Quantidade de cartas a comprar. Use inteiro positivo e valide contra remaining.
cards	String CSV	Depende	Lista de codigos, por exemplo AS,0H,KD. Usado em baralho parcial, pilhas, compra seletiva e retorno.
remaining	bool	Nao	Em /shuffle/, true embaralha apenas as cartas ainda no monte principal.
jokers_enabled	bool	Nao	Em /new/, true inclui dois curingas.
pile_name	String (path)	Nas operacoes de pilha	Nome dinamico da pilha, por exemplo player, dealer ou discard.

DECISAO FLUTTER

Construa URLs com `Uri.https` ou `Uri.parse(...).replace(queryParameters: ...)`. Nao concatene parametros fornecidos pelo usuario diretamente.

Exemplo de requisicao

```
GET https://deckofcardsapi.com/api/deck/new/shuffle/?deck_count=6

{
  "success": true,
  "deck_id": "yq7lyyxeu1va",
  "remaining": 312,
  "shuffled": true
}
```

O exemplo acima combina seis decks: 6 x 52 = 312 cartas. O identificador ilustrativo muda a cada criacao.

3 Catalogo de endpoints

#	Endpoint	Funcao	Resposta principal
1	/api/deck/new/shuffle/?deck_count={n}	Criar baralho embaralhado	DeckResponse
2	/api/deck/new/	Criar baralho novo ordenado	DeckResponse
3	/api/deck/new/shuffle/?cards={csv}	Criar baralho parcial	DeckResponse
4	/api/deck/{deck_id}/draw/?count={n}	Comprar cartas do monte	DrawResponse
5	/api/deck/{deck_id}/shuffle/	Reembaralhar o baralho	DeckResponse
6	/api/deck/{deck_id}/shuffle/?remaining=true	Embaralhar somente o restante	DeckResponse
7	/api/deck/{deck_id}/pile/{pile}/add/?cards={csv}	Adicionar cartas a uma pilha	PileResponse
8	/api/deck/{deck_id}/pile/{pile}/shuffle/	Embaralhar uma pilha	PileResponse
9	/api/deck/{deck_id}/pile/{pile}/list/	Listar cartas da pilha	PileResponse
10	/api/deck/{deck_id}/pile/{pile}/draw/?count={n}	Comprar do topo da pilha	PileDrawResponse
11	/api/deck/{deck_id}/pile/{pile}/draw/bottom/?count={n}	Comprar do fundo da pilha	PileDrawResponse
12	/api/deck/{deck_id}/pile/{pile}/draw/random/?count={n}	Comprar aleatoriamente	PileDrawResponse
13	/api/deck/{deck_id}/return/[?cards={csv}]	Devolver cartas compradas	PileResponse/Deck Response
14	/api/deck/{deck_id}/pile/{pile}/return/[?cards={csv}]	Devolver cartas da pilha	PileResponse
15	/static/img/back.png	Obter imagem do verso	image/png

OBSERVACAO

As operacoes de pilha sao documentadas como incompativeis com baralhos compostos por varios decks. Isso conflita com o uso tipico de seis decks no Blackjack. Para seis decks, mantenha as maos localmente no Flutter e use apenas create/draw/shuffle; ou utilize um unico deck se quiser pilhas remotas.

Recursos estaticos de imagem

Cada carta retorna a URL principal em `image` e, normalmente, as variantes em `images.png` e `images.svg`. O verso padrao esta em `https://deckofcardsapi.com/static/img/back.png`. No Flutter mobile, PNG e a opcao mais simples com `Image.network`; SVG exige um pacote como flutter_svg.

4 Criacao, compra e reembaralhamento

4.1 Criar baralho embaralhado

GET `/api/deck/new/shuffle/?deck_count=1`

Finalidade: Cria um identificador e deixa as cartas prontas para compra em ordem aleatoria.

Parametros: deck_count (opcional, padrao 1). Para Blackjack tradicional, use 6 se pilhas remotas nao forem necessarias.

Retorno: success, deck_id, shuffled e remaining.

4.2 Criar baralho novo, nao embaralhado

GET `/api/deck/new/?jokers_enabled=true`

Finalidade: Cria um deck em ordem conhecida: espadas, ouros, paus e copas. Pode incluir dois curingas.

Parametros: jokers_enabled (opcional). Sem curingas, remaining inicia em 52.

Retorno: DeckResponse com shuffled=false.

4.3 Criar baralho parcial

GET `/api/deck/new/shuffle/?cards=AS,2S,KS,AD,2D,KD`

Finalidade: Cria um baralho apenas com os codigos informados. E util para testes de regra e demonstracoes reproduziveis.

Parametros: cards: codigos separados por virgula. O exemplo oficial usa valores A,2..9,0,J,Q,K e naipes S,D,C,H.

Retorno: DeckResponse; remaining e igual ao numero de cartas validas no conjunto.

4.4 Comprar cartas

GET `/api/deck/{deck_id}/draw/?count=2`

Finalidade: Remove cartas do topo e as devolve na lista cards.

Parametros: count define a quantidade. Substituir deck_id por new cria um deck embaralhado e compra na mesma chamada.

Retorno: DrawResponse com cards e remaining atualizado.

Resposta real validada durante a elaboracao


```
{
  "success": true,
  "deck_id": "yq7lyyxeu1va",
  "cards": [
    {
      "code": "9H",
      "image": "https://deckofcardsapi.com/static/img/9H.png",
      "images": {"svg": ".../9H.svg", "png": ".../9H.png"},
      "value": "9",
      "suit": "HEARTS"
    }
  ],
  "remaining": 50
}
```

4.5 Reembaralhar

GET `/api/deck/{deck_id}/shuffle/`

Finalidade: Embaralha novamente o baralho identificado sem criar um novo deck_id.

Parametros: Sem parametro obrigatorio; a API ja conhece a quantidade de decks associada ao ID.

Retorno: DeckResponse com shuffled=true e a contagem atualizada.

4.6 Reembaralhar apenas o monte principal

GET `/api/deck/{deck_id}/shuffle/?remaining=true`

Finalidade: Embaralha somente as cartas ainda disponiveis no monte principal, preservando cartas compradas ou em pilhas fora dele.

Parametros: remaining=true.

Retorno: DeckResponse com shuffled=true.

Significado do ciclo de vida

Momento	Estado esperado	Acao do app
Inicio da sessao	Sem deck_id	Criar deck e guardar deck_id.
Distribuicao	remaining diminui	Adicionar as cartas retornadas as maos locais.
Nova rodada	Cartas anteriores ainda estao fora do monte	Criar novo deck, devolver cartas ou seguir comprando, conforme a regra escolhida.
Corte/limite	remaining abaixo do limite definido	Criar outro shoe ou reembaralhar conforme as regras do jogo.
Sessao expirada	deck_id pode nao existir apos duas semanas sem atividade	Descartar o ID e criar um novo deck.

ESTADO O deck_id e parte do estado da partida, nao uma configuracao global. Salve-o no controller/provider da mesa e invalide-o quando iniciar uma nova sessao.

Exemplo de compra em uma unica chamada

```
GET /api/deck/new/draw/?count=4

// Uso indicado para prototipos: cria um deck embaralhado e compra 4 cartas.
// Guarde o deck_id retornado para as proximas compras.
```

5 Operacoes com pilhas

Pilhas sao colecoes nomeadas criadas sob demanda. Podem representar a mao do jogador, a mao do dealer ou descarte. Primeiro compre as cartas do monte; depois use seus codigos para adiciona-las a uma pilha. A propria documentacao alerta que pilhas nao funcionam com multiplos decks.

5.1 Adicionar cartas

GET `/api/deck/{deck_id}/pile/{pile_name}/add/?cards=AS,2S`

Finalidade: Cria a pilha se necessario e move para ela cartas que ja foram compradas. Cartas podem ser movidas entre pilhas.

Parametros: pile_name no path e cards na query.

Retorno: PileResponse com remaining do monte e contagem das pilhas.

5.2 Embaralhar pilha

GET `/api/deck/{deck_id}/pile/{pile_name}/shuffle/`

Finalidade: Randomiza a ordem interna da pilha indicada.

Parametros: deck_id e pile_name.

Retorno: PileResponse com estado resumido das pilhas.

5.3 Listar cartas

GET `/api/deck/{deck_id}/pile/{pile_name}/list/`

Finalidade: Consulta cartas e contagem de pilhas.

Parametros: deck_id e pile_name. A resposta pode incluir outras pilhas conhecidas no objeto piles.

Retorno: PileResponse; a pilha consultada inclui cards, enquanto outras podem aparecer apenas com remaining.

Resposta de pilha validada


```
{
  "success": true,
  "deck_id": "yq7lyyxeu1va",
  "remaining": 50,
  "piles": {
    "player": {
      "remaining": 2,
      "cards": [
        {"code": "9H", "value": "9", "suit": "HEARTS", "image": "..."},
        {"code": "3C", "value": "3", "suit": "CLUBS", "image": "..."}
      ]
    }
  }
}
```

5.4 Compra e retorno em pilhas

Comprar do topo

GET `/api/deck/{deck_id}/pile/{pile_name}/draw/?count=2`

Finalidade: Retira cartas do topo da pilha. Em vez de count, cards=AS pode selecionar codigos especificos.

Parametros: count ou cards.

Retorno: PileDrawResponse: cards retiradas, estado de piles e remaining do monte principal.

Comprar do fundo

GET `/api/deck/{deck_id}/pile/{pile_name}/draw/bottom/?count=2`

Finalidade: Retira do fundo da pilha.

Parametros: count opcional conforme o exemplo oficial.

Retorno: PileDrawResponse.

Comprar aleatoriamente

GET `/api/deck/{deck_id}/pile/{pile_name}/draw/random/?count=2`

Finalidade: Retira cartas aleatorias da pilha.

Parametros: count opcional conforme o exemplo oficial.

Retorno: PileDrawResponse.

Devolver cartas compradas

GET `/api/deck/{deck_id}/return/?cards=AS,2S`

Finalidade: Devolve ao monte principal cartas que estavam fora de pilhas. Sem cards, devolve todas as cartas elegiveis.

Parametros: cards opcional.

Retorno: Resposta com success, deck_id, shuffled/remaining e, quando aplicavel, piles.

Devolver cartas de uma pilha

GET `/api/deck/{deck_id}/pile/{pile_name}/return/?cards=AS,2S`

Finalidade: Devolve ao monte principal cartas da pilha indicada. Sem cards, devolve todas as cartas da pilha.

Parametros: pile_name e cards opcional.

Retorno: PileResponse com contagens atualizadas.

SEMANTICA	remaining no nivel raiz sempre representa o monte principal. piles[nome].remaining representa a quantidade dentro daquela pilha. Nao some os dois para decidir quantas cartas podem ser compradas diretamente do monte.

6 Dicionario de dados das respostas

Campo JSON	Tipo recomendado	Presenca	Significado no app
success	bool	Todas as respostas JSON documentadas	Indica se a operacao foi aceita. Ainda verifique o status HTTP.
deck_id	String	Respostas de baralho/pilha	Identificador remoto do baralho. Chave para chamadas posteriores.
shuffled	bool?	Criacao/shuffle/return	Informa que o baralho foi embaralhado. Nao aparece em todas as operacoes.
remaining	int	Nivel raiz	Quantidade de cartas disponiveis no monte principal depois da operacao.
cards	List<PlayingCard>	Compras e listagem/compra de pilha	Cartas devolvidas pela operacao. Pode estar ausente em respostas resumidas.
piles	Map<String, PileState>	Operacoes de pilha/retorno	Mapa dinamico: a chave e o nome da pilha.
piles.*.remaining	int	Cada pilha	Quantidade de cartas naquela pilha. A documentacao antiga mostra numero ou texto; normalize ambos.
piles.*.cards	List<PlayingCard>?	Listagem ou compra	Cartas conhecidas daquela pilha; pode ser omitido em resumos.
code	String	Cada carta	Codigo compacto, por exemplo AS, 0H ou KD.
image	String/Uri	Cada carta	URL principal da imagem PNG da carta.
images	CardImages?	Cada carta nas respostas atuais	Variantes svg e png. Mantenha opcional para compatibilidade.
images.svg	String/Uri	Dentro de images	URL da frente em SVG.
images.png	String/Uri	Dentro de images	URL da frente em PNG.
value	String	Cada carta	Valor legivel: ACE, 2..10, JACK, QUEEN ou KING.
suit	String	Cada carta	Naipe: SPADES, DIAMONDS, CLUBS ou HEARTS.

NORMALIZACAO Converta remaining por uma funcao que aceite int, num ou String numerica. Os exemplos oficiais de listagem de pilhas exibem strings, enquanto respostas atuais testadas retornaram inteiros.

Tipos de resposta sugeridos

Tipo Dart	Usado por	Campos centrais
DeckResponse	new, shuffle, return	success, deckId, remaining, shuffled, piles?
DrawResponse	draw do monte	success, deckId, cards, remaining
PileResponse	add, shuffle, list, return	success, deckId, remaining, piles
PileDrawResponse	draw de pilha	success, deckId, remaining, piles, cards
PlayingCard	cards	code, image, images, value, suit
PileState	piles[nome]	remaining, cards?

7 Codigos de cartas e interpretacao

O parametro cards usa um codigo de dois caracteres. O primeiro representa o valor e o segundo representa o naipe. A comparacao pode ser feita em maiusculas, embora a documentacao informe que os codigos de entrada nao diferenciam maiusculas de minusculas.

Parte	Codigos	Significado
Valor	A	As
Valor	2,3,4,5,6,7,8,9	Cartas numericas
Valor	0	Dez - o codigo compacto usa zero
Valor	J,Q,K	Valete, dama e rei
Naipe	S	Spades - espadas
Naipe	D	Diamonds - ouros
Naipe	C	Clubs - paus
Naipe	H	Hearts - copas

Conversao para pontuacao de Blackjack

value retornado	Pontos iniciais	Regra
2 a 10	Valor numerico	Conversao direta.
JACK, QUEEN, KING	10	Todas as figuras valem dez.
ACE	11 ou 1	Comece com 11 e reduza 10 enquanto a soma for maior que 21.

```
int blackjackScore(List<PlayingCard> cards) {
  var total = 0;
  var aces = 0;
  for (final card in cards) {
    switch (card.value) {
      case 'ACE': aces++; total += 11;
      case 'KING' || 'QUEEN' || 'JACK': total += 10;
      default: total += int.parse(card.value);
    }
  }
  while (total > 21 && aces > 0) { total -= 10; aces--; }
  return total;
}
```

REGRA DO APP

O calculo acima e derivado do Blackjack e nao vem da API. Variacoes de mesa, split, soft 17, double e apostas devem ser modeladas separadamente.

8 Estruturas Dart para o Flutter

Os DTOs abaixo absorvem o contrato JSON sem acoplar a interface ao formato bruto. Eles usam construtores imutaveis, fromJson e uma conversao tolerante para contagens. Em um projeto maior, os mesmos modelos podem ser gerados com json_serializable/freezed.

```
int _asInt(Object? value, {int fallback = 0}) {
  if (value is int) return value;
  if (value is num) return value.toInt();
  return int.tryParse(value?.toString() ?? '') ?? fallback;
}

class CardImages {
  const CardImages({required this.svg, required this.png});
  final Uri svg;
  final Uri png;

  factory CardImages.fromJson(Map<String, dynamic> json) => CardImages(
    svg: Uri.parse(json['svg'] as String),
    png: Uri.parse(json['png'] as String),
  );
}

class PlayingCard {
  const PlayingCard({
    required this.code,
    required this.image,
    required this.value,
    required this.suit,
    this.images,
  });
  final String code;
  final Uri image;
  final CardImages? images;
  final String value;
  final String suit;

  factory PlayingCard.fromJson(Map<String, dynamic> json) => PlayingCard(
    code: json['code'] as String,
    image: Uri.parse(json['image'] as String),
    images: json['images'] is Map
        ? CardImages.fromJson(Map<String, dynamic>.from(json['images'] as Map))
        : null,
    value: json['value'] as String,
    suit: json['suit'] as String,
  );
}
```

Por que Uri?

Usar `Uri` nos modelos reduz concatenacao insegura e deixa explicito que o campo representa um recurso remoto. Caso a equipe prefira modelos puramente serializaveis, mantenha String no DTO e converta para Uri na camada de apresentacao.

8.1 Respostas e pilhas

```
class DeckResponse {
  const DeckResponse({
    required this.success,
    required this.deckId,
    required this.remaining,
    this.shuffled,
    this.piles = const {},
  });
  final bool success;
  final String deckId;
  final int remaining;
  final bool? shuffled;
  final Map<String, PileState> piles;

  factory DeckResponse.fromJson(Map<String, dynamic> json) => DeckResponse(
    success: json['success'] == true,
    deckId: json['deck_id'] as String,
    remaining: _asInt(json['remaining']),
    shuffled: json['shuffled'] as bool?,
    piles: _parsePiles(json['piles']),
  );
}

class DrawResponse {
  const DrawResponse({
    required this.success,
    required this.deckId,
    required this.cards,
    required this.remaining,
  });
  final bool success;
  final String deckId;
  final List<PlayingCard> cards;
  final int remaining;

  factory DrawResponse.fromJson(Map<String, dynamic> json) => DrawResponse(
    success: json['success'] == true,
    deckId: json['deck_id'] as String,
    cards: (json['cards'] as List? ?? const [])
        .map((e) => PlayingCard.fromJson(Map<String, dynamic>.from(e as Map)))
        .toList(growable: false),
    remaining: _asInt(json['remaining']),
  );
}
```

8.2 Modelos de pilha e estado do jogo

```
class PileState {
  const PileState({required this.remaining, this.cards = const []});
  final int remaining;
  final List<PlayingCard> cards;

  factory PileState.fromJson(Map<String, dynamic> json) => PileState(
    remaining: _asInt(json['remaining']),
    cards: (json['cards'] as List? ?? const [])
        .map((e) => PlayingCard.fromJson(Map<String, dynamic>.from(e as Map)))
        .toList(growable: false),
  );
}

Map<String, PileState> _parsePiles(Object? raw) {
  if (raw is! Map) return const {};
  return raw.map((key, value) => MapEntry(
    key.toString(),
    PileState.fromJson(Map<String, dynamic>.from(value as Map)),
  ));
}

class PileResponse {
  const PileResponse({
    required this.success,
    required this.deckId,
    required this.remaining,
    required this.piles,
    this.cards = const [],
  });
  final bool success;
  final String deckId;
  final int remaining;
  final Map<String, PileState> piles;
  final List<PlayingCard> cards;
}
```

Modelo de dominio separado

```
enum GamePhase { idle, dealing, playerTurn, dealerTurn, finished }

class BlackjackHand {
  const BlackjackHand(this.cards);
  final List<PlayingCard> cards;
  int get score => blackjackScore(cards);
  bool get isBust => score > 21;
  bool get isBlackjack => cards.length == 2 && score == 21;
}

class BlackjackGameState {
  const BlackjackGameState({
    required this.deckId,
    required this.remaining,
    required this.player,
    required this.dealer,
    required this.phase,
  });
  final String deckId;
  final int remaining;
  final BlackjackHand player;
  final BlackjackHand dealer;
  final GamePhase phase;
}
```

SEPARACAO	DTOs espelham a API; modelos de dominio expressam regras do Blackjack. Essa separacao evita espalhar chaves JSON e regras de pontuacao pela interface.

9 Cliente HTTP e tratamento de erros

Exemplo usando o pacote http. O cliente recebe a dependencia no construtor, o que permite substitui-la por MockClient nos testes. Todos os endpoints devem compartilhar timeout, validacao de status e decodificacao defensiva.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class DeckApiException implements Exception {
  const DeckApiException(this.message, {this.statusCode});
  final String message;
  final int? statusCode;
}

class DeckOfCardsApi {
  DeckOfCardsApi(this._client);
  final http.Client _client;
  static const _host = 'deckofcardsapi.com';

  Future<Map<String, dynamic>> _get(
    List<String> path, [Map<String, String>? query]
  ) async {
    final uri = Uri.https(_host, path.join('/'), query);
    final response = await _client.get(uri)
        .timeout(const Duration(seconds: 10));
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw DeckApiException('Falha HTTP', statusCode: response.statusCode);
    }
    final body = jsonDecode(response.body);
    if (body is! Map<String, dynamic>) {
      throw const DeckApiException('JSON inesperado');
    }
    if (body['success'] == false) {
      throw DeckApiException(body['error']?.toString() ?? 'Operacao recusada');
    }
    return body;
  }

  Future<DeckResponse> createShuffledDeck({int deckCount = 6}) async {
    final json = await _get(
      ['api', 'deck', 'new', 'shuffle', ''],
      {'deck_count': '$deckCount'},
    );
    return DeckResponse.fromJson(json);
  }

  Future<DrawResponse> draw(String deckId, {int count = 1}) async {
    final json = await _get(
      ['api', 'deck', deckId, 'draw', ''],
      {'count': '$count'},
    );
    return DrawResponse.fromJson(json);
  }
}
```

9.1 Metodos adicionais do repositorio

```
Future<DeckResponse> reshuffle(
  String deckId, {bool onlyRemaining = false}
) async {
  final json = await _get(
    ['api', 'deck', deckId, 'shuffle', ''],
    onlyRemaining ? {'remaining': 'true'} : null,
  );
  return DeckResponse.fromJson(json);
}

Future<PileResponse> addToPile(
  String deckId, String pileName, List<String> codes
) async {
  final json = await _get(
    ['api', 'deck', deckId, 'pile', pileName, 'add', ''],
    {'cards': codes.join(',')},
  );
  return PileResponse.fromJson(json);
}

Future<PileResponse> listPile(String deckId, String pileName) async {
  final json = await _get(
    ['api', 'deck', deckId, 'pile', pileName, 'list', ''],
  );
  return PileResponse.fromJson(json);
}
```

Politica de falhas

Falha	Tratamento recomendado
Timeout/sem rede	Exibir acao de tentar novamente. Nao repetir automaticamente uma compra sem reconciliar o estado, pois a primeira requisicao pode ter sido processada.
HTTP fora de 2xx	Criar DeckApiException com statusCode e registrar corpo de forma segura.
success=false	Tratar como falha de negocio mesmo se o HTTP for 200.
JSON invalido/campo ausente	Falhar de modo controlado, registrar o contrato recebido e nao atualizar o estado da mesa.
deck_id expirado	Criar novo deck e reiniciar a sessao; nao reutilizar maos com um baralho diferente.
remaining insuficiente	Bloquear a compra ou iniciar um novo shoe antes de chamar a API.
Imagem indisponivel	Mostrar placeholder/verso local e manter o jogo funcional com value/suit.

IDEMPOTENCIA Comprar cartas altera estado. Um retry cego pode comprar cartas extras. Se houver timeout apos envio, consulte o estado disponivel ou reinicie a rodada de forma explicita; nao duplique a chamada automaticamente.

10 Fluxo sugerido para o Blackjack

Etapa	Chamada API	Estado Flutter
1. Nova mesa	new/shuffle/?deck_count=6	Guardar deckId e remaining=312.
2. Distribuicao inicial	draw/?count=4	Alternar cartas entre player e dealer; ocultar a carta fechada apenas na UI.
3. Hit do jogador	draw/?count=1	Adicionar a player.cards e recalcular score.
4. Turno do dealer	draw/?count=1 enquanto a regra exigir	Adicionar a dealer.cards e recalcular score.
5. Resultado	Nenhuma chamada obrigatoria	Comparar scores, bust e blackjack localmente.
6. Proxima rodada	Continuar draw ou criar/reembaralhar	Limpar maos, manter ou substituir deckId conforme politica da mesa.

Arquitetura recomendada

Camada	Responsabilidade
DeckOfCardsApi	HTTP, timeout, status, JSON bruto e DTOs.
BlackjackRepository	Orquestrar criacao/compra, persistir deck_id da sessao e mapear erros.
BlackjackController / Cubit / Notifier	Aplicar comandos do usuario e produzir estados imutaveis para a UI.
Modelos de dominio	Pontuacao, fase, vencedor, apostas e regras da mesa.
Widgets	Renderizar cartas, botoes e feedback; nao fazer chamadas HTTP diretamente.

Decisao sobre pilhas

Configuracao	Recomendacao
Blackjack com 6 decks	Nao usar pilhas remotas, pois a documentacao declara incompatibilidade com multiplos decks. Guarde player/dealer hands localmente e use draw.
Demo com 1 deck	Pilhas player/dealer podem espelhar as maos no servidor, desde que o app continue tratando pontuacao e regras.
Testes previsiveis	Use baralho parcial com cards conhecidos; isso facilita validar Blackjack, bust e ases flexiveis.

SEGURANCA VISUAL

Para a carta fechada do dealer, mantenha a carta real no estado e renderize back.png ate a fase apropriada. Nao e necessario um endpoint especial para carta oculta.

11 Testes, cuidados e referencias

Checklist de aceite

- Criar um deck e comprovar que deck_id nao esta vazio e remaining corresponde a configuracao.
- Comprar N cartas e comprovar cards.length == N e reducao de remaining.
- Desserializar remaining quando vier como int e como String.
- Desserializar uma carta mesmo quando images estiver ausente.
- Calcular corretamente ases flexiveis: A+9+A = 21; A+K = Blackjack; K+Q+2 = bust.
- Tratar timeout sem retry automatico de draw.
- Exibir placeholder quando a imagem falhar.
- Criar novo deck quando o ID estiver expirado ou invalido.
- Impedir uso de pilhas remotas quando deck_count > 1.

Cuidados de producao

Tema	Recomendacao
Disponibilidade	A documentacao publica nao apresenta SLA. Trate o servico como dependencia externa e mantenha estados de loading/erro.
Persistencia	Persistir deck_id so enquanto fizer sentido retomar a sessao. IDs inativos sao eliminados apos duas semanas.
Privacidade	Nao envie dados pessoais: a API precisa apenas de IDs, quantidades, codigos e nomes de pilha.
Logs	Registrar endpoint, status e deck_id mascarado/parcial; nunca depender do log para reconstruir a ordem secreta do deck.
Compatibilidade	Manter parsing tolerante para campos opcionais e contagens int/String, mas falhar em campos essenciais como deck_id.
Flutter Web	Validar no navegador-alvo a politica CORS atual do servico. Essa verificacao nao e necessaria para apps Flutter nativos.

Fontes consultadas

1. Deck of Cards API - documentacao oficial: https://deckofcardsapi.com/ (acesso em 21/09/2026).

2. Repositorio oficial do projeto, licenca MIT e orientacoes de execucao: https://github.com/crobertsbmw/deckofcards (acesso em 21/09/2026).

3. Exemplos reais validados diretamente na API durante a elaboracao: criacao de deck, compra de duas cartas, adicao e listagem da pilha player. IDs e cartas exibidos sao ilustrativos e temporarios.

CONCLUSAO

A API cobre adequadamente o baralho e os recursos visuais. Uma implementacao Flutter robusta deve manter o contrato HTTP isolado, controlar regras e pontuacao localmente e evitar pilhas remotas ao usar multiplos decks.